2. Estructuras de control. Arrays

T4. Códigos de barras e calendario

En esta tarea, se sigue manualmente un bucle `while` y se prueban y corrigen dos métodos con bucles, uno de ellos con bucles anidados y una instrucción `break`. A continuación, se escribe un método que lee datos con un bucle `do-while`. Las técnicas empleadas son las de U2A4 y U2A5: tabla de trazas, patrones de repetición, bucles `for`, bucles anidados, sentencias de salto y pruebas de bucle. También se utilizan las técnicas de prueba de U2A1 y U2A3.

Comienza con este archivo, `U2T4_usuario.java`. `readCode` aún no lee nada: devuelve 0, y el programa termina de leer sin hacer ninguna pregunta. Se completa en 4_f. `isLeapYear` y `daysInMonth` son de U2T2 y solo las usa la ampliación.

```java {.compact}
import java.util.Locale;
import java.util.Scanner;
/**
 * U2T4. Códigos de barras e calendario.
 *
 * @author usuario
 */
class BarcodesAndCalendar {
    /**
     * Comproba un código EAN-13.
     *
     * @param code código de 13 díxitos, sen os ceros á esquerda
     * @return true se o código está no intervalo e o seu díxito de
     *         control é correcto
     */
    public static boolean isValidEan(long code) {
        int sum = 0;
        int position = 0;
        while (code > 0) {
            int digit = (int) (code % 10);
            if (position % 2 == 0) {
                sum += 3 * digit;
            } else {
                sum += digit;
            }
            code = code / 10;
            position++;
        }
        return sum % 10 == 0;
    }
```

```
/**
 * Amosa se un código é válido e se coincide co esperado.
 *
 * @param code     código que se proba
 * @param expected resultado que debería devolver isValidEan
 */
public static void checkEan(long code, boolean expected) {
    boolean result = isValidEan(code);
    String verdict = result == expected ? "OK" : "ERRO";
    System.out.println(code + " -> " + result
            + " [" + verdict + "]");
}
/**
 * Escribe o calendario dun mes, cunha liña por semana.
 *
 * @param days  número de días do mes, de 28 a 31
 * @param first día da semana do día 1, de 1 (luns) a 7 (domingo)
 */
public static void printMonth(int days, int first) {
    System.out.println(" Lu Ma Mé Xo Ve Sá Do");
    int day = 1;
    for (int week = 1; week <= 5; week++) {
        for (int weekday = 1; weekday <= 7; weekday++) {
            if (day > days) {
                break;
            }
            if (week == 1 && weekday < first) {
                System.out.print("   ");
            } else {
                System.out.printf("%3d", day);
                day++;
            }
        }
        System.out.println();
    }
}
public static long readCode(Scanner scanner) {
    return 0;   // provisional ata 4_f
}
// Da U2T2: só os usa a ampliación, 4_g
/**
 * Indica se un ano é bisesto no calendario gregoriano.
 *
 * @param year ano que se comproba
 * @return true se o ano é bisesto
 */
```

```
public static boolean isLeapYear(int year) {
    return year % 4 == 0
            && (year % 100 != 0 || year % 400 == 0);
}
/**
 * Calcula o número de días dun mes.
 *
 * @param month número do mes, de 1 a 12
 * @param year  ano, que só inflúe en febreiro
 * @return o número de días, ou -1 se o mes non é válido
 */
public static int daysInMonth(int month, int year) {
    return switch (month) {
        case 1, 3, 5, 7, 8, 10, 12 -> 31;
        case 4, 6, 9, 11 -> 30;
        case 2 -> isLeapYear(year) ? 29 : 28;
        default -> -1;
    };
}
public static void main(String[] args) {
    checkEan(12345678905L, true);
    System.out.println();
    printMonth(31, 4);
    System.out.println("---");
    System.out.println();
    // Coma decimal en calquera computador (U2A3)
    Locale.setDefault(Locale.of("gl", "ES"));
    Scanner scanner = new Scanner(System.in);
    int checked = 0;
    int valid = 0;
    long code = readCode(scanner);
    while (code != 0) {
        checked++;
        if (isValidEan(code)) {
            System.out.println("Válido");
            valid++;
        } else {
            System.out.println("Non válido");
        }
        code = readCode(scanner);
    }
    scanner.close();
    if (checked == 0) {
        System.out.println("Non se comprobou ningún código");
    } else {
        System.out.printf(
```

```
                "Comprobados: %d. Válidos: %d (%.1f %%)%n",
                checked, valid, valid * 100.0 / checked);
        }
    }
}
```

Muchos productos de supermercado tienen un código de barras EAN-13 de 13 dígitos, y los ISBN de 13 dígitos de los libros también son códigos EAN-13. El último dígito, el de la derecha, es el dígito de control. Se calcula a partir de los otros doce y se utiliza para detectar si un código se ha leído o escrito incorrectamente. La función `isValidEan` debe cumplir con esta especificación:
- El código llega en un formato `long`, que no guarda los ceros de la izquierda: el código `0012345678905` llega como 12345678905. Por lo tanto, cualquier valor entre 1 y 9 999 999 999 999 es un código de 13 dígitos, completado con ceros a la izquierda.
- Un valor fuera de este intervalo no es un código válido.
- Los dígitos se numeran desde la derecha a izquierda, y el dígito de control es el que da la posición 1. Cada dígito en una posición impar se multiplica por 1, y cada dígito en una posición par se multiplica por 3.
- El código es válido si la suma de estos productos es un múltiplo de 10.

El método `main` inicial ya comprueba un caso: `isValidEan(12345678905L)` debe devolver `true`.

`printMonth` escribe el calendario de un mes y debe cumplir con esta especificación:

- Recibe el número de días del mes, de 28 a 31, y el día de la semana del día 1, de 1 (lunes) a 7 (domingo). No comprueba los valores.
- Escribe una línea de cabecera y luego una línea por semana, de lunes a domingo. Dependiendo de los días y el día de la semana del día 1, un mes ocupa de cuatro a seis semanas.
- Cada día ocupa tres caracteres, con el número alineado a la derecha debajo de su día de la semana. Antes del día 1 hay tres espacios por cada día de la semana anterior.
- Después del último día del mes, la línea termina. No hay líneas en blanco.

Octubre de 2026 tiene 31 días y comienza un jueves, por lo que `printMonth(31, 4)` debería escribir:
```text
 Lu Ma Mé Xo Ve Sá Do
           1  2  3  4
  5  6  7  8  9 10 11
 12 13 14 15 16 17 18
 19 20 21 22 23 24 25
 26 27 28 29 30 31
```

Tarea 4_a
En esta sección, seguirás a mano el bucle `isValidEan` y compararás su traza con la ejecución.

1. Sin ejecutar el programa, cree la tabla de traza para `isValidEan(96385074)` en el documento. Toma una fila con los valores iniciales y una por iteración, con los valores de `code`, `digit`, `position` y `sum` al final de la iteración, después de `position++`.

Iteración	code	digit	position	sum
inicial	96385074		0	0
1	9638507	4	1	12
2	963850	7	2	19
3	96385	0	3	19
4	9638	5	4	24
5	963	8	5	48
6	96	3	6	51
7	9	6	7	69
8	0	9	8	78

2. Indique en el documento qué devuelve el método, según su tabla.

return sum % 10 == 0;
78 % 10 == 0; (el resto es 8) false

3. Agregue esta sentencia temporal al final del cuerpo del bucle, después de `position++`:

```java
System.out.println(code + " " + digit + " " + position
        + " " + sum);
```

4. Reemplace la llamada a `checkEan` en `main` con esta línea:

```java
System.out.println(isValidEan(96385074));
```

5. Ejecute el programa y pegue en el documento las líneas que escriben la sentencia temporal y el resultado.

9638507 4 1 12
963850 7 2 19
96385 0 3 19
9638 5 4 24
963 8 5 48
96 3 6 51
9 6 7 69
0 9 8 78
false

6. Compare esas líneas con tu tabla e indique las diferencias, si las hay, en el documento.

Las salidas son las mismas que la tabla y el return.
No hay diferencias.

7. Calcule manualmente, utilizando la regla de especificación, si 96385074 es un código válido y escriba el cálculo en el documento.

posición	13	12	11	10	9	8	7	6	5	4	3	2	1
code	0	0	0	0	0	9	6	3	8	5	0	7	4
multiplica	1	3	1	3	1	3	1	3	1	3	1	3	1
resultado	0	0	0	0	0	27	6	9	8	15	0	21	4
1	0	0	0	0	0	0	0	0	0	0	0	0	7
3	1	3	1	3	1	3	1	3	1	3	1	3	1

27+6+9+8+15+0+21+4=90 / 90%10=0 / Es múltiplo de 10.
96385074 es un código válido

8. Indique en el documento si su cálculo coincide con el resultado que devuelve el método.

No coincide mi calculo da 90 que es múltiplo de 10, pero el método da 78 que no es múltiplo de 10 y da false.

9. Si no coincide, indique en el documento en qué fila de la tabla se observa la causa.

Se observa ya desde la fila 1 porque el resultado de ambos no coincide, en el del método da 12 y en el calculado por la especificación da 4.
El método empieza con posición 0 que es par y le multiplica por 3; mientras que el calculado empieza en posición 1 que es impar y lo multiplica por 1.

10. Deshaga los dos cambios temporales: elimine la sentencia del bucle y vuelva a colocar la llamada a `checkEan` en `main`.
No modifique `isValidEan` hasta 4_c.

Tarea 4_b
En esta sección, se prueba la función `isValidEan` con una tabla de casos y se localizan las filas que fallan.

La tabla de prueba de `isValidEan` combina las clases de equivalencia y los valores límite de U2A1 con los casos de número de iteración de U2A4:

Fila	Qué comprueba	Código

1	Valor típico	`8412345678905`
2	Dígito de control incorrecto	`8412345678906`
3	Ceros iniciales	`12345678905`
4	Ceros iniciales, ocho dígitos	`96385074`
5	Una iteración	`5`
6	Código válido más bajo	`17`
7	0: cero iteraciones, fuera de rango	`0`
8	Negativo: cero iteraciones, fuera de rango	`-17`
9	Código válido más alto	`9999999999994`
10	Catorce dígitos, suma múltiplo de 10	`10000000000007 `

1. Reemplace la llamada a `checkEan` en `main` por una llamada por fila. El valor esperado de cada una sale de la especificación y se escribe antes de ejecutar. Un literal entero que no cabe en un `int` requiere el sufijo `L` (U1A4).

2. Ejecute el programa y pegue las líneas de prueba en el documento.

8412345678905 -> false [ERRO]
8412345678906 -> false [OK]
12345678905 -> true [OK]
96385074 -> false [ERRO]
5 -> false [OK]
17 -> false [ERRO]
0 -> true [ERRO]
-17 -> true [ERRO]
9999999999994 -> false [ERRO]
10000000000007 -> false [OK]

3. Indique en el documento qué filas dan `ERROR`.

Fila	**code**	**expected**
1	checkEan(8412345678905L, true);	8412345678905 -> false [ERRO]
4	checkEan(96385074, true);	96385074 -> false [ERRO]
6	checkEan(17, true);	17 -> false [ERRO]
7	checkEan(0, false);	0 -> true [ERRO]
8	checkEan(-17, false);	-17 -> true [ERRO]
9	checkEan(9999999999994L, true);	9999999999994 -> false [ERRO]

Dan ERRO las filas 1,4,6,7,8 y 9

Tarea 4_c

En esta sección, corriges `isValidEan` hasta que todas las filas de 4_b devuelvan `OK`. El código original esta en el fichero de partida, y el corregido, en el que entregas: el documento solo incluye lo que no se ve en el código.

1. Corrige en `isValidEan` uno de los errores que pusieron en evidencia las filas de 4_b.
2. Ejecuta el programa con todas las filas.

8412345678905 -> true [OK]
8412345678906 -> false [OK]
12345678905 -> true [OK]
96385074 -> true [OK]
5 -> false [OK]
17 -> true [OK]
0 -> true [ERRO]
-17 -> true [ERRO]
9999999999994 -> true [OK]
10000000000007 -> true [ERRO]

3. Escribe en el documento qué filas pusieron en evidencia ese error y por qué fallaron.

Las filas 1, 4, 6 y 9 lo ponen en evidencia.

El método empieza con posición 0 que es par y le multiplica por 3; mientras que el calculado empieza en posición 1 que es impar y lo multiplica por 1. (el método debe empezar en 1 para cumplir con la especificación y que todos los valores se calculen correctamente con su posición)

4. Repite los pasos 1 a 3 hasta que todas las filas den `OK`.

8412345678905 -> true [OK]
8412345678906 -> false [OK]
12345678905 -> true [OK]
96385074 -> true [OK]
5 -> false [OK]
17 -> true [OK]
0 -> false [OK]
-17 -> false [OK]
9999999999994 -> true [OK]
10000000000007 -> false [OK]

NÚMEROS FUERA DE RANGO
Las filas 7, 8 y 10 lo ponen en evidencia.
El método admite números fuera del rango porque no ha puesto una condición que los de valor false antes de llegar al while para que no entren, dando por validos los números fuera de rango.

5. Indica en el documento si alguna fila pasó de `OK` a `ERROR` durante las correcciones.

Si.
Al cambiar int position = 0; por int position = 1; cambió
10000000000007 -> true [ERRO]

6. Si alguna pasó, responde en el documento: ¿por qué pasó de `OK` a `ERROR`?

Porque con la posición 0 da 22, que no es múltiplo de 10 y da false pero por casualidad, si lo calculas con posición 1, da 10 que es múltiplo de 10 y da true porque el método no prueba el rango. Incluyendo al método el rango se arregla y ya da OK.

7. Pega la salida final de las pruebas en el documento.

8412345678905 -> true [OK]
8412345678906 -> false [OK]
12345678905 -> true [OK]
96385074 -> true [OK]
5 -> false [OK]
17 -> true [OK]
0 -> false [OK]
-17 -> false [OK]
9999999999994 -> true [OK]
10000000000007 -> false [OK]

Tarea 4_d

En esta sección probarás la función `printMonth` con seis meses reales.

Estos meses son las filas de la tabla de prueba `printMonth`:

Fila	Mes	Días	Día 1	Semanas	Último día
1	Octubre 2026	31	Jueves (4)		
2	Junio 2026	30	Lunes (1)		
3	Febrero 2026	28	Domingo (7)		
4	Febrero 2027	28	Lunes (1)		
5	Agosto 2026	31	Sábado (6)		
6	Noviembre 2026	30	Domingo (7)		

1. Antes de ejecutar el programa, observe cada mes en un calendario, como el del reloj de Windows o el de su teléfono. Anote en su documento cuántas semanas abarca y en qué día de la semana termina.

Fila	**Mes**	**Días**	**Día 1**	**Semanas**	**Último día**
1	Octubre 2026	31	Jueves (4)	5	Sábado (6)
2	Junio 2026	30	Lunes (1)	5	Martes (2)
3	Febrero 2026	28	Domingo (7)	5	Sábado (6)

4	Febrero 2027	28	Lunes (1)	4	Domingo (7)
5	Agosto 2026	31	Sábado (6)	6	Lunes (1)
6	Noviembre 2026	30	Domingo (7)	6	Lunes (1)

2. En `main`, reemplace la llamada a `printMonth` y la línea `---` que la sigue con una llamada por fila, cada una seguida de su propia línea `---`.
3. Ejecute el programa y pegue los calendarios en su documento.

```
Lu Ma Mé Xo Ve Sá Do
       1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
Lu Ma Mé Xo Ve Sá Do
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30
Lu Ma Mé Xo Ve Sá Do
            1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28
Lu Ma Mé Xo Ve Sá Do
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
```

```
Lu Ma Mé Xo Ve Sá Do
            1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
Lu Ma Mé Xo Ve Sá Do
               1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
```

4. Indique en su documento qué filas no dan los resultados esperados.

Fila 1
OCTUBRE 2026
Empieza en miércoles en vez de jueves y el domingo deja espacio en blanco(lo salta) y pone el 5 en el lunes.
Fila 3
FEBRERO 2026
Empieza en viernes en vez de domingo y salta el sábado y domingo y pone el 2 en el lunes siguiente
Fila 4
FEBRERO 2027
Esta correcto el calendario pero hay un salto de linea al final.
Fila 5
AGOSTO 2026
El 1 y el 2 están descentrados, salta el sábado y domingo y pone el 3 en el lunes.
Falta el día 31.
Fila 6
NOVIEMBRE 2026
Empieza en viernes en vez de domingo, y salta el sabado y el domingo, poniendo el 2 en el lunes.
Falta el día 30.

5. Describa en su documento la diferencia que observa en cada una de ellas.

- Día 1 en lugar equivocado
  Fila 1,3,5 y 6
- Falta último día
  Fila 5 y 6
- Salto de linea

Fila 4
No modifique `printMonth` hasta 4_e.

Tarea 4_e

En esta sección, corrija `printMonth` hasta que todas las filas de 4_d den el resultado esperado. Al igual que en 4_c, el documento solo toma lo que no se ve en el código.

1. Corrige uno de los errores en `printMonth` que pusieron en evidencia las filas de 4_d.
2. Ejecuta el programa con todas las filas.
3. Escribe en el documento qué filas revelaron ese error y por qué fallaron.

El error de que empezara el día 1 en un lugar equivocado se debía a que el print de huecos vacios anteriores al primer día solo tenía dos huecos

```
if (week == 1 && weekday < first) {
   System.out.print("  ");
```

y el formato imprime el número con 3 por lo que todo se descuadraba a la hora de empezar el día.
Lo revelaban las filas 1, 3, 5 y 6 al empezar en lugar equivocado.
Añadiendo un espacio se arregla.

4. Repite los pasos 1 a 3 hasta que todas las filas den el resultado esperado.

La falta del último dia se debe a que solo se ha puesto 5 semanas de máximo cuando hay meses que necesitan 6, cambiando el número lo corrige.
for (int week = 1; week <= 6; week++)
Lo revelan las filas 5 y 6.

Para que no de vuelta de más el for de fuera; y no imprima lineas vacias con el println, hay que ponerle una restricción para que salga del for si el day>days pues ya no habría que hacer nada más en el código.
Sin ella vuelve a entrar hasta que week <=6 y estando ya todos los días.
Se consigue añadiendo la condición y el break a este.

```
for (int week = 1; week <= 6; week++) {
   if (day > days) {
      break;
```

Lo revelan las filas 1,2,3 y 4.

5. Indica en el documento si alguna fila dejó de dar el resultado esperado durante las correcciones.

Si, al cambiar week <=6, las filas 1,2 y 3 que estaban bien han añadido un salto de linea al final y la fila 4 que tenía uno ha pasado a tener dos.

6. Si alguna dejó de darlo, responde en el documento: ¿por qué dejó de dar el resultado esperado?

Porque al cambiar week <= 5 a 6 hace que se de una vuelta más en el for de fuera, haciendo que en los meses que ya han acabado salga en el break y pase por println imprimiendo una linea vacia.

7. Pega el resultado final de los calendarios en el documento.

```
 Lu Ma Mé Xo Ve Sá Do
           1  2  3  4
  5  6  7  8  9 10 11
 12 13 14 15 16 17 18
 19 20 21 22 23 24 25
 26 27 28 29 30 31
 Lu Ma Mé Xo Ve Sá Do
  1  2  3  4  5  6  7
  8  9 10 11 12 13 14
 15 16 17 18 19 20 21
 22 23 24 25 26 27 28
 29 30
 Lu Ma Mé Xo Ve Sá Do
                    1
  2  3  4  5  6  7  8
  9 10 11 12 13 14 15
 16 17 18 19 20 21 22
 23 24 25 26 27 28
 Lu Ma Mé Xo Ve Sá Do
  1  2  3  4  5  6  7
  8  9 10 11 12 13 14
 15 16 17 18 19 20 21
 22 23 24 25 26 27 28
 Lu Ma Mé Xo Ve Sá Do
                 1  2
  3  4  5  6  7  8  9
 10 11 12 13 14 15 16
 17 18 19 20 21 22 23
 24 25 26 27 28 29 30
 31
 Lu Ma Mé Xo Ve Sá Do
                    1
  2  3  4  5  6  7  8
  9 10 11 12 13 14 15
 16 17 18 19 20 21 22
 23 24 25 26 27 28 29
 30
```

Tarea 4_f

En esta sección se escribe `readCode`, que lee los códigos del teclado. El `main` ya lo utiliza en un bucle con centinela y escribe el resumen de los códigos comprobados. `readCode` debe cumplir esta especificación:

- Haz la pregunta `"Código (0 para finalizar): "`, con un espacio después de los dos puntos, y devuelve el número leído. El 0 finaliza la lectura de `main`.
- Si lo que se escribe no es un número entero que quepa en un `long`, escribe `Entrada no válida` y vuelve a hacer la pregunta.

- Una línea vacía no cuenta: el programa continúa esperando. Si hay algo más en la línea después del número, se descarta, como en U2A3.
- Ninguna línea que se escriba puede detener el programa con una excepción.

Estas son las filas de la tabla de prueba:

| Cola | Qué comprueba | Entradas | Esperado|
|:----:| ---------- | ---------- | -------- | |
1 | Varios códigos | `8412345678905`, `8412345678906`, `96385074` y `0` | |
| 2 | Texto | `abc`, `17` y `0` | |
| 3 | Entero que no cabe en un `long` | `99999999999999999999` y `0` | | |
4 | Línea vacía | Enter, `17` y `0` | |
| 5 | Número seguido de texto | `84 123` y `0` | |

1. Escribe el cuerpo de `readCode` y documéntalo con un comentario de documentación, como `isValidEan`.
2. Antes de ejecutarlo, completa el documento con la columna que esperas que el programa escriba en cada fila.
3. Ejecuta el programa con las entradas de cada fila, una por línea, y pega cada salida de la primera pregunta en el documento.

Tarea 4_g

Esta sección es una ampliación: se realiza después de terminar las anteriores. En ella se calcula el día de la semana del primer día del mes y se elabora el calendario de febrero de 2028 con ese dato.

`firstWeekday(int month, int year)` devuelve el día de la semana del primer día de un mes, desde el 1 (lunes) hasta el 7 (domingo). Solo se llama con meses del 1 al 12 y años a partir del 2000, y se calcula de la siguiente manera:

- El 1 de enero de 2000 fue sábado.
- Se cuentan los días que han pasado desde esa fecha hasta el 1 del mes:
  - 366 por cada año bisiesto anterior al mes;
  - 365 por cada uno de los otros años anteriores;
  - los días de cada mes anterior del mismo año.
- Si `total` es ese número de días, el día de la semana es `(total + 5) % 7 + 1`.

1. Escribe `firstWeekday` con bucles `for` y los métodos `isLeapYear` y `daysInMonth` en el archivo, y documéntalo con un comentario de documentación.
2. Escribe `checkWeekday(int month, int year, int expected)`, que muestra el resultado y si coincide con el esperado, como `checkEan`.
3. Dibuja la tabla de prueba para `firstWeekday` en el documento. Usa los casos de prueba de los bucles U2A4 para cada uno de los dos bucles y los límites del resultado. Los esperados se toman de un calendario real.
4. Agregue a `main` una llamada a `checkWeekday` para cada fila de su tabla.
5. Ejecute el programa y pegue la salida de prueba en el documento.
6. Agregue a `main`, antes de leer los códigos, las líneas que escriben el calendario de febrero de 2028 con `printMonth`, `firstWeekday` y `daysInMonth`.
7. Ejecute el programa y pegue el calendario en el documento.

Entrega

Dos archivos sueltos:

- `U2T4_usuario.java`, la clase `BarcodesAndCalendar` completa: `isValidEan` y `printMonth` corregidos, `readCode` y `main` con las pruebas de 4_b y 4_d y la lectura de los códigos, sin la sentencia temporal de 4_a. Si hizo 4_g, también `firstWeekday`, `checkWeekday`, sus pruebas y el calendario de febrero de 2028.
- `U2T4_usuario.pdf`, con lo que pide cada paso, con su número:
  - la tabla de traza, la salida y el cálculo de 4_a;
  - las salidas con los métodos originales y las filas que fallan;
  - para cada corrección de 4_c y 4_e, las filas que lo mostraron y por qué fallaron;
  - las tablas de 4_d y 4_f con las esperadas;
  - las salidas finales y las respuestas.

Los esperados de 4_b van en el código, en las llamadas a `checkEan`, y no en el documento.